You need to save them all to the database because of how Rails handles associations—specifically has_many through: associations such as the relationship between projects and users in this example. If the associations were just ordinary has_many associations, Rails would be able to manage the two-way relationship in memory without touching the database (at least Rails 4 can; older versions don't do that). However, when there is a join relationship denoted by has_many: through, Rails internals will always need to have saved objects with IDs to resolve the relationship. Although you can work around this problem (and I'll discuss some workarounds when I talk about fast tests in Chapter 16, *Running Tests Faster and Running Faster Tests*, on page 319), for the moment the workarounds are more complicated and distracting than the straightforward creation of the data.

A user can view a project in which he or she has a role. This method in User will make the tests pass:

security/02/app/models/user.rb

```
def can_view?(project)
  project.in?(projects)
end
```

Although the unit test passes, the integration test still fails because you haven't added the access check into the controller. That brings in the second kind of logic you need to add: does the controller successfully block the page if the user doesn't have access? In some cases I might write a unit test for this behavior; for example, a workflow or service object might help determine whether the code redirects and to where, but in this case I think I'll let the existing integration tests cover the logic.

To make this test pass, you need to incorporate the check of can_view? into the controller. You can make this work in a couple of ways—here's one:

security/02/app/controllers/projects_controller.rb

```
def show
  @project = Project.find(params[:id])
  unless current_user.can_view?(@project)
    redirect_to new_user_session_path
    return
  end
  respond_to do |format|
    format.html {}
    format.js { render json: @project.as_json(root: true, include: :tasks) }
  end
end
```

This controller method redirects to the login screen and returns if the user is blocked; otherwise it continues normally.